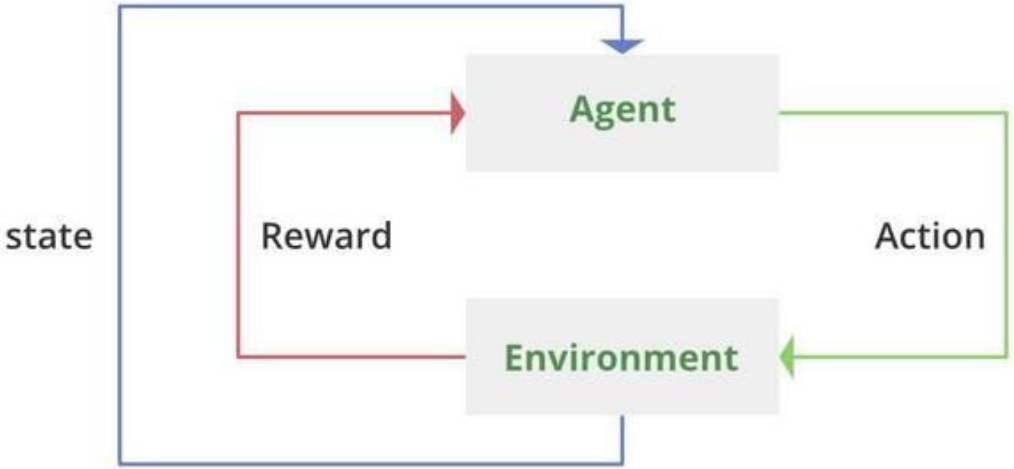
Chapter 6 .
Reinforcement Learning

Ph.D. Nguyen Thi Khanh Tien

Reinforcement learning (RL) is a machine learning technique where an agent learns to make decisions by interacting with an environment. Unlike supervised learning, which provides labeled data, RL agents learn from the consequences of their actions.

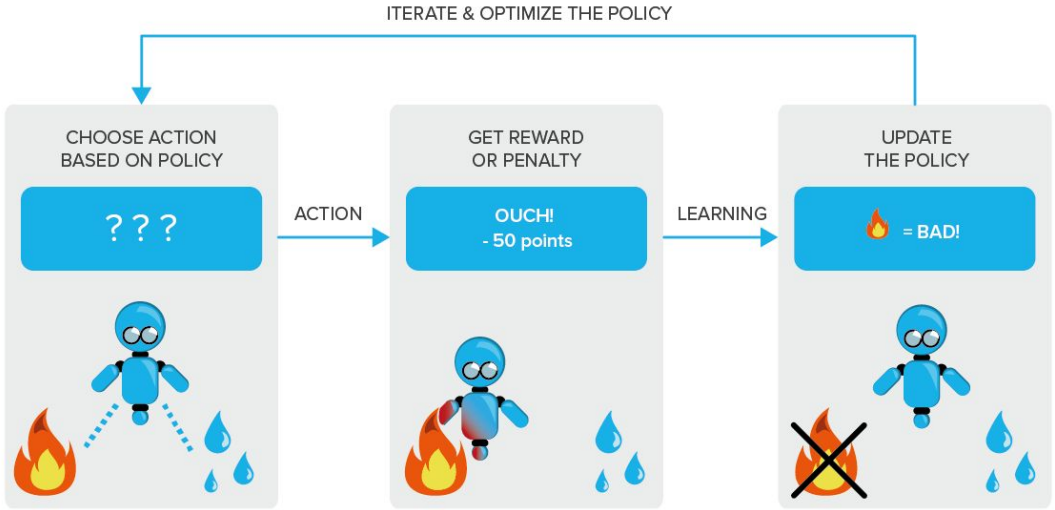
Core Components of Reinforcement Learning

- **Agent:** The decision-maker, who learns to interact with the environment.
- **Environment:** The world in which the agent operates.
- **State:** The current situation or condition of the environment.
- **Action:** The choices the agent can make.
- **Reward:** A feedback signal indicating the success or failure of an action.
- **Policy:** The agent's strategy for selecting actions given a state.



How Reinforcement Learning Works

1. **Initialization:** The agent starts in an initial state of the environment.
2. **Action Selection:** The agent chooses an action based on its current policy.
3. **Environment Interaction:** The agent takes the action, and the environment transitions to a new state.
4. **Reward:** The agent receives a reward based on the transition.
5. **Learning:** The agent updates its policy based on the received reward.
6. **Repeat:** The process is repeated iteratively, with the agent gradually improving its decision-making.



Difference between Reinforcement learning and Supervised learning

Reinforcement learning	Supervised learning
Reinforcement learning is all about making decisions sequentially. In simple words, we can say that the output depends on the state of the current input and the next input depends on the output of the previous input	In Supervised learning, the decision is made on the initial input or the input given at the start
In Reinforcement learning decision is dependent, So we give labels to sequences of dependent decisions	In supervised learning the decisions are independent of each other so labels are given to each decision.
Example: Chess game,text summarization	Example: Object recognition,spam detetction

Types of Reinforcement Learning

There are two types of RL

1. **Positive:** Positive Reinforcement is defined as when an event, occurs due to a particular behavior, increases the strength and the frequency of the behavior. In other words, it has a positive effect on behavior.

   Advantages of reinforcement learning are:

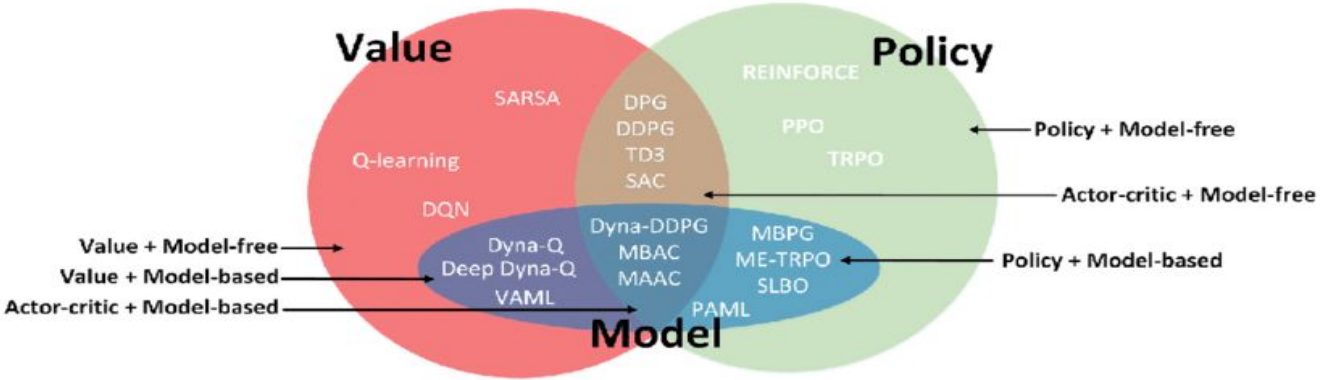
   - Maximizes Performance
   - Sustain Change for a long period of time
   - Too much Reinforcement can lead to an overload of states which can diminish the results

2. **Negative:** Negative Reinforcement is defined as strengthening of behavior because a negative condition is stopped or avoided.

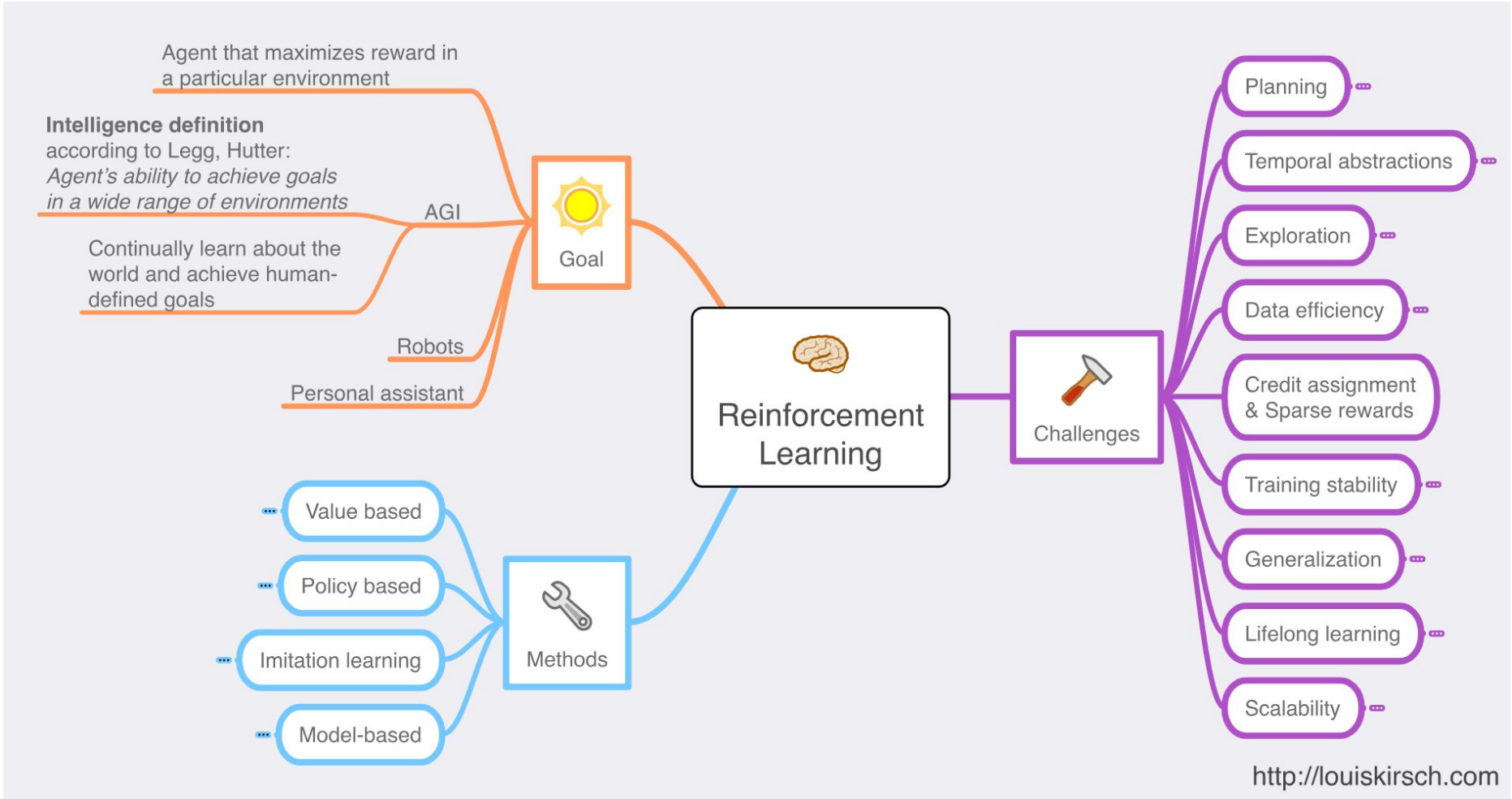
   Advantages of reinforcement learning:

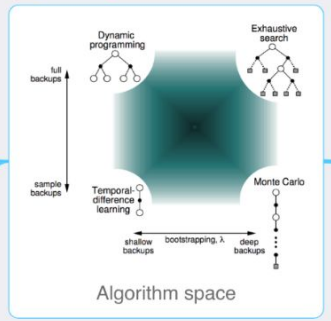
   - Increases Behavior
   - Provide defiance to a minimum standard of performance
   - It Only provides enough to meet up the minimum behavior

Type of Reinforcement Learning



- ***Value-Based Methods***: These methods estimate the value of each state or state-action pair in the environment. The agent learns which actions are better by comparing the values. For example, Q-learning and Deep Q-Networks (DQN)
- ***Policy-Based Methods:*** In these methods, the agent directly learns the policy that maps states to the probability of taking each action. This approach is helpful in high-dimensional or continuous action spaces. Examples include REINFORCE and policy gradient methods.
- ***Model-Based Methods:*** These methods involve building a model of the environment. The agent learns how the environment behaves and makes decisions using this model. This approach can be more sample efficient but requires a good model of the environment.


Agent that maximizes reward in a particular environment
Intelligence definition according to Legg, Hutter: Agent's ability to achieve goals in a wide range of environments
AGI
Continually learn about the world and achieve human-defined goals
Robots
Personal assistant
Goal
Reinforcement Learning
Value based
Policy based
Imitation learning
Model-based
Methods
Challenges
Planning
Temporal abstractions
Exploration
Data efficiency
Credit assignment & Sparse rewards
Training stability
Generalization
Lifelong learning
Scalability
http://louiskirsch.com

- Methods
  - Value based
    - Implementations
      - Q-Learning
      - Sarsa
      - Dyna
      - DDPG
    - 

      Algorithm space
      - Full backups vs sample backups (tree width)
      - Shallow backups vs deep backups (tree depth)
    - Advantages
      - More data efficient than policy based RL
    - Disadvantages
      - High dimensional / continuous action spaces
      - Stochastic policies
  - Policy based
    - Policy gradient
      - REINFORCE (Vanilla)
      - TRPO
      - PPO
      - A2C
      - A3C
      - Generalized Advantage Estimation (GAE)
      - TRPO Kronecker-factored (ACKTR)
      - ACER
      - SAC
    - Deterministic Policy Gradients
      - DPG
      - DDPG
      - TD3
    - Evolutionary
      - CEM
      - CMA-ES
      - PEPG
      - NES
      - ES
    - Policy search
      - PILCO
      - REPS
  - Imitation learning
    - DAGGER
    - GAIL
    - DeepMimic
    - VAIL
    - MetaMimic
  - Model-based
    - Pure planning
    - Expert Iteration
    - Data Augmentation
    - Embedding planning loops into policies
    - AIXI

http://louiskirsch.com

Applications of Reinforcement Learning

- **Robotics:** Controlling robots to perform tasks like grasping objects or navigating environments.
- **Game Playing:** Developing agents to play games like chess, Go, and video games.
- **Finance:** Algorithmic trading, portfolio management.
- **Healthcare:** Personalized treatment plans, drug discovery.
- **Recommendation Systems:** Suggesting products or content to users.

Key Concepts

- **Exploration vs. Exploitation:** The agent must balance exploring new actions to discover better rewards with exploiting known good actions to maximize immediate rewards.
- **Markov Decision Process (MDP):** A mathematical framework for modeling sequential decision-making problems.
- **Value Function:** Estimates the expected future reward from a given state.
- **Q-Learning:** A popular RL algorithm that learns the expected future reward for taking an action in a given state.
- **Deep Q-Networks (DQN):** Combines deep learning with Q-learning for complex problems.

Challenges and Future Directions

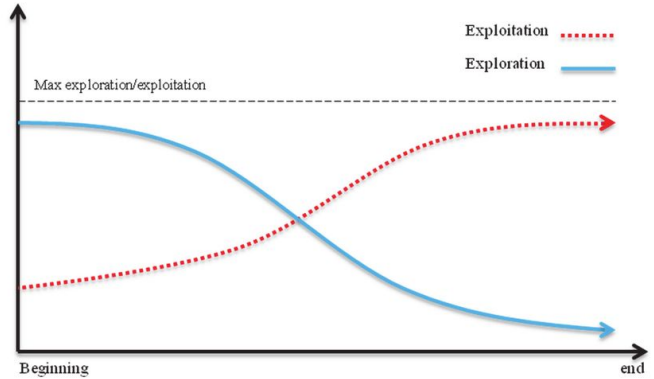
While reinforcement learning has shown impressive results, there are still challenges to overcome:

- **Sample Efficiency:** RL algorithms often require a large amount of data to learn effectively.
- **Exploration-Exploitation Dilemma:** Finding the right balance between exploring new actions and exploiting known good actions.
- **Credit Assignment:** Determining which actions contributed to a reward can be difficult.

Future research focuses on developing more efficient algorithms, addressing sample efficiency, and applying RL to real-world problems with greater complexity.

Exploration vs. Exploitation: The Balancing Act

Exploration vs. exploitation is a fundamental challenge in reinforcement learning. It's about striking the right balance between trying new things (exploration) and sticking with what works (exploitation).

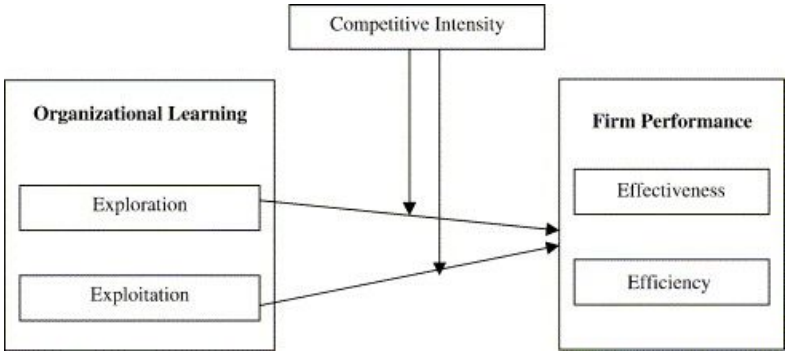


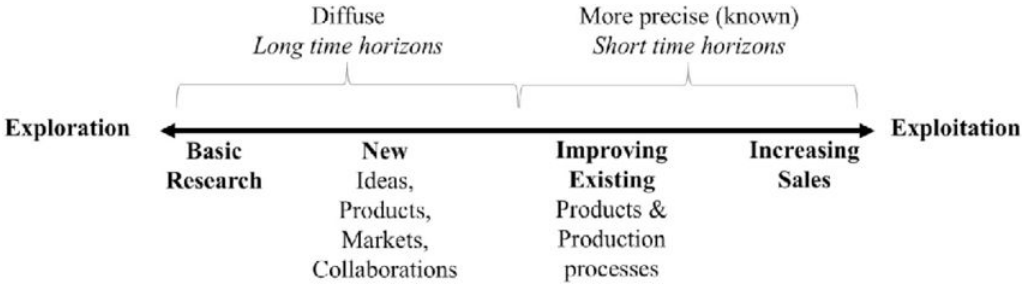
Exploration

- **Definition:** Trying out new actions or strategies to discover potentially better options.
- **Goal:** To find optimal or near-optimal solutions by exploring the environment.
- **Risk:** Potentially lower short-term rewards due to uncertainty.

Exploitation

- **Definition:** Using the current best-known strategy to maximize immediate rewards.
- **Goal:** To maximize performance based on current knowledge.
- **Risk:** Getting stuck in a local optimum and missing out on better possibilities.





The Dilemma

An agent that only exploits its current knowledge will never discover better options. Conversely, an agent that only explores might never achieve significant rewards. The key lies in finding the right balance between the two.

Strategies for Balancing Exploration and Exploitation

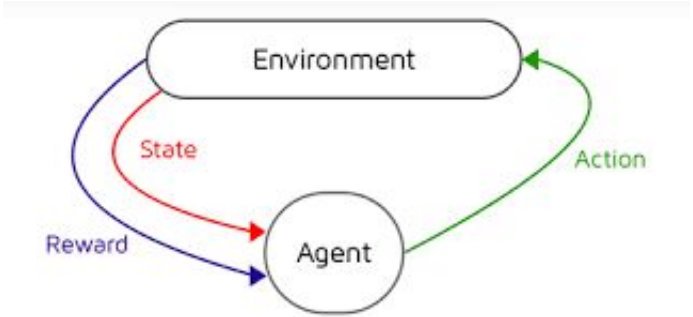
Several techniques can help an agent navigate this dilemma:

- **Epsilon-greedy:** A simple approach where the agent chooses a random action with probability epsilon and the best-known action with probability 1-epsilon.
- **Upper Confidence Bound (UCB):** This method balances exploration and exploitation by considering both the average reward and the number of times an action has been tried.
- **Thompson Sampling:** Treats the problem as a Bayesian inference problem, assigning probabilities to different actions and selecting actions based on these probabilities.
- **Reinforcement Learning Algorithms:** Many RL algorithms inherently balance exploration and exploitation, such as Deep Q-Networks (DQN) and Policy Gradient methods.

Example: A Restaurant Recommendation System

- **Exploration:** Recommending a new restaurant to a user, even if the user's past preferences suggest a different type of cuisine.
- **Exploitation:** Recommending a restaurant similar to the user's previous favorites.

Markov Decision Process (MDP)



A Markov Decision Process (MDP) is a mathematical framework for modeling decision-making in situations where outcomes are partly random and partly under the control of a decision maker. It's a cornerstone in reinforcement learning.

Components of an MDP

- **States (S):** All possible situations or conditions the agent can be in.
- **Actions (A):** The set of possible actions the agent can take in a given state.
- **Transition Probabilities (P):** The probability of moving from one state to another after taking a specific action.
- **Rewards (R):** The immediate value received by the agent for transitioning from one state to another and taking a specific action.
- **Policy (π):** A policy is a strategy that specifies the action to be taken in each state. A policy can be deterministic, where a specific action is chosen for each state, or stochastic, where a probability distribution over actions is defined for each state.
- **Discount factor (γ):** A value between 0 and 1 that determines the importance of future rewards.

States:	**S**
Model:	**T(S, a, S') ~ P(S' | S, a)**
Actions:	**A(S), A**
Reward:	**R(S), R(S, a), R(S, a, S')**
Policy:	**$\prod(S) \rightarrow a$** **$\prod^*$**

Markov Decision Process

Solving a Markov Decision Process

There are several methods for solving MDPs, including:

- **Value Iteration:** An iterative algorithm that updates the value of each state under the assumption of acting optimally from that state onwards. The algorithm iteratively updates the value of each state until the values converge to a stable set, which represents the value function of the optimal policy.
- **Policy Iteration:** An iterative algorithm that alternates between policy evaluation (computing the value of a policy) and policy improvement (using the value function to find a better policy). This process is repeated until the policy converges to the optimal policy.
- **Q-Learning:** A model-free reinforcement learning algorithm that learns the quality of actions (denoted as Q-values) directly from experience without a model of the environment's dynamics. Q-learning can find the optimal policy even when the model of the environment is unknown.

Applications of Markov Decision Processes

MDPs have a broad range of applications, including:

- Robotics for path planning and obstacle avoidance.
- Resource management in computer systems and networks.
- Financial models for investment and portfolio optimization.
- Game theory and decision-making in competitive environments.
- Medical treatment planning and patient care optimization.

A **State** is a set of tokens that represent every state that the agent can be in.

A **Model** (sometimes called Transition Model) gives an action's effect in a state. In particular, T(S, a, S') defines a transition T where being in state S and taking an action 'a' takes us to state S' (S and S' may be the same). For stochastic actions (noisy, non-deterministic) we also define a probability P(S'|S,a) which represents the probability of reaching a state S' if action 'a' is taken in state S. Note Markov property states that the effects of an action taken in a state depend only on that state and not on the prior history.

Action A is a set of all possible actions. A(s) defines the set of actions that can be taken being in state S.

A **Reward** is a real-valued reward function. R(s) indicates the reward for simply being in the state S. R(S,a) indicates the reward for being in a state S and taking an action 'a'. R(S,a,S') indicates the reward for being in a state S, taking an action 'a' and ending up in a state S'.

A **Policy** is a solution to the Markov Decision Process. A policy is a mapping from S to a. It indicates the action 'a' to be taken while in state S.

States:	**S**
Model:	**T(S, a, S') ~ P(S' | S, a)**
Actions:	**A(S), A**
Reward:	**R(S), R(S, a), R(S, a, S')**
Policy:	**$\Pi(S) \rightarrow a$** **Π^***

Markov Decision Process

Example of a grid world

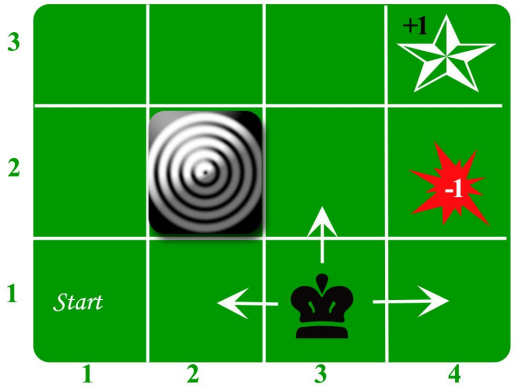
An agent lives in the grid (ex: 3*4).

The grid has a START state(grid(1,1)).

The purpose of the agent is to wander around the grid to finally reach the Blue Diamond (grid (4,3)).

Under all circumstances, the agent should avoid the Fire grid (orange color, grid (4,2)). Also, the grid(2,2) is a blocked grid, it acts as a wall hence the agent cannot enter it.

The agent can take any one of these actions: **UP, DOWN, LEFT, RIGHT**



Walls block the agent's path, i.e., if there is a wall in the direction the agent would have taken, the agent stays in the same place. So for example, if the agent says LEFT in the START grid he would stay put in the START grid.

First Aim: To find the shortest sequence getting from START to the Diamond. Two such sequences can be found:

- **RIGHT- RIGHT- UP- UP-RIGHT**
- **UP- UP- RIGHT- RIGHT- RIGHT**

Let us take the second one (UP UP RIGHT RIGHT RIGHT) for the subsequent discussion.
The move is now noisy. 80% of the time the intended action works correctly. 20% of the time the action agent takes causes it to move at right angles.

For example, if the agent says UP the probability of going UP is 0.8 whereas the probability of going LEFT is 0.1, and the probability of going RIGHT is 0.1 (since LEFT and RIGHT are right angles to UP).

The agent receives rewards for each time step:-

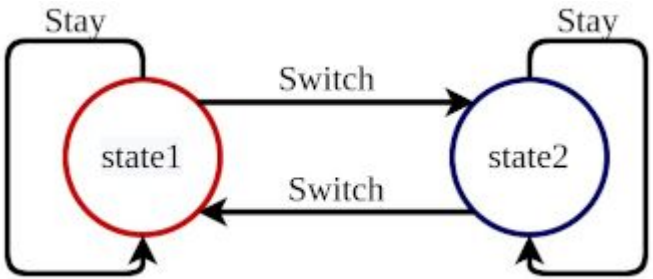
- Small reward for each step (can be negative when can also be term as punishment, in the above example entering the Fire can have a reward of -1).
- Big rewards come at the end (good or bad).
- The goal is to Maximize the sum of rewards.

Simple MDP coding example - Two state MDP

This is an example on how to implement a two-state Markov Decision Process (MDP). It has only two states: state 1 and state 2. Applications MDPs often appear in machine learning and reinforcement learning.

At any stage, the agent can take any of the two actions. The actions are to stay or switch. The switch will change the current state to the other one and stay won't make any changes.

At any point in time, we can print out the current state of the system. The state transition diagram can be visualized as below.




```python
class TwoStateMDP(object):
    """docstring for TwoState"""
    def __init__(self):
        self.states = 2
        self.nactions = 2
        self.currentS = 1

    def stay(self):
        pass

    def toggle(self):
        if self.currentS ==1:
            self.currentS=2
        else:
            self.currentS=1

switch = TwoStateMDP()

print(switch.currentS)
switch.toggle()
print(switch.currentS)
switch.toggle()
print(switch.currentS)
```

The command *'switch = TwoStateMDP()'* generate an object of the two-state MDP type. By default, it will be of state 1. When we call the *'toggle()'* function, it gets switched to the other state.

Value Function in Reinforcement Learning

A value function in reinforcement learning is an estimate of how good it is for an agent to be in a particular state or to take a particular action in a given state. In simpler terms, it's a prediction of the expected future reward that an agent can obtain from that state or action.

There are two primary types of value functions: State Value function (V), Action Value Function (Q)

- The value function $V^{\pi}(s)$ assigns each state the expected reward

$$V^{\pi}(s) = \mathop{\mathbb{E}}_{a_t, a_{t+i}, s_{t+i}} \left[\sum_{i=0}^{\infty} \gamma^i r_{t+i} | s_t = s \right]$$

- Usually not informative enough to make decisions.
- The Q-value $Q^{\pi}(s, a)$ is the expected reward of taking action a in state s and then continuing according to π.

$$Q^{\pi}(s, a) = \mathop{\mathbb{E}}_{a_{t+i}, s_{t+i}} \left[\sum_{i=0}^{\infty} \gamma^i r_{t+i} | s_t = s, a_t = a \right]$$

The Role of Value Functions in Reinforcement Learning

- **Decision Making:** Agents use value functions to decide which actions to take. They choose actions that lead to states with higher expected returns.
- **Learning:** Value functions are learned through experience. Reinforcement learning algorithms update value function estimates based on the agent's interactions with the environment.
- **Policy Improvement:** By estimating the value of different states and actions, agents can improve their policies over time.

Key Concepts Related to Value Functions

- **Discount Factor (γ):** A value between 0 and 1 that determines the importance of future rewards. A higher discount factor gives more weight to future rewards.
- **Bellman Equation:** A fundamental equation in reinforcement learning that relates the value of a state to the expected values of its successor states.
- **Dynamic Programming:** A mathematical technique used to solve the Bellman equation and compute optimal value functions.
- **Temporal Difference Learning:** A reinforcement learning method that learns value functions directly from experience without relying on a complete model of the environment.

Bellman equations

- The foundation of many RL algorithms

$$
\begin{aligned}
V^{\pi}(s) &= \mathop{\mathbb{E}}_{a_t, a_{t+i}, s_{t+i}} \left[\sum_{i=0}^{\infty} \gamma^i r_{t+i} | s_t = s \right] \\
&= \mathop{\mathbb{E}}_{a_t} [r_t | s_t = s] + \gamma \mathop{\mathbb{E}}_{a_t, a_{t+i}, s_{t+i}} \left[\sum_{i=1}^{\infty} \gamma^i r_{t+i+1} | s_t = s \right] \\
&= \mathop{\mathbb{E}}_{a_t} [r_t | s_t = s] + \gamma \mathop{\mathbb{E}}_{s_{t+1}} [V^{\pi}(s_{t+1}) | s_t = s] \\
&= \sum_{a,r} P^{\pi}(a|s_t) p(r|a, s_t) \cdot r + \gamma \sum_{a,s'} P^{\pi}(a|s_t) p(s'|a, s_t) \cdot V^{\pi}(s')
\end{aligned}
$$

- Similar equation holds for Q

$$
\begin{aligned}
Q^{\pi}(s, a) &= \mathop{\mathbb{E}}_{a_{t+i}, s_{t+i}} \left[\sum_{i=0}^{\infty} \gamma^i r_{t+i} | s_t = s, a_t = a \right] \\
&= \sum_{r} p(r|a, s_t) \cdot r + \gamma \sum_{s'} p(s'|a, s_t) \cdot V^{\pi}(s') \\
&= \sum_{r} p(r|a, s_t) \cdot r + \gamma \sum_{a', s'} p(s'|a, s_t) p(a'|s') \cdot Q^{\pi}(s', a')
\end{aligned}
$$

Optimal Bellman equations

- First step is understand the Bellman equation for the optimal policy π^*
- Under this policy $V^*(s) = \max_a Q^*(s, a)$

$$
\begin{aligned}
V^*(s) &= \max_a \left[\mathbb{E}\left[r_{t+1} | s_t = s, a_t = a \right] + \gamma \mathop{\mathbb{E}}_{s_{t+1}} \left[V^*(s_{t+1}) | s_t = s, a_t = a \right] \right] \\
&= \max_a \left[\sum_r p(r|a, s_t) \cdot r + \gamma \sum_{s'} p(s'|a, s_t) \cdot V^*(s') \right] \\
Q^*(s, a) &= \mathbb{E}\left[r_{t+1} | s_t = s, a_t = a \right] + \gamma \mathop{\mathbb{E}}_{s_{t+1}} \left[\max_{a'} Q^*(s_{t+1}, a') | s_t = s, a_t = a \right] \\
&= \sum_r p(r|a, s_t) \cdot r + \gamma \sum_{s'} p(s'|a, s_t) \cdot \max_{a'} Q^*(s', a')
\end{aligned}
$$

- Set on nonlinear equations.
- Same issues as before.

Q-Learning: Learning from Actions

- Q-learning is a simple algorithm to find the optimal policy without knowing the model.
- Q^* is the unique solution to the optimal Bellman equation.

$$Q^*(s,a) = \mathbb{E}\left[r_{t+1}|s_t = s, a_t = a\right] + \gamma \mathop{\mathbb{E}}_{s_{t+1}}\left[\max_{a'} Q^*(s_{t+1}, a')|s_t = s, a_t = a\right]$$

- We don't know the model and don't want to update all states simultaneously.
- Solution - given sample $s_t, a_t, r_{t+1}, s_{t+1}$ from the environment update your Q-values so they are closer to satisfying the bellman equation.
  - off-policy method: Samples don't have to be from the optimal policy.
- Samples need to be diverse enough to see everything - exploration.

- Given Q-value the best thing we can do (given our limited knowledge) is to take $a = \arg\max_{a'} Q(s, a')$ - exploitation
- How do we balance exploration with exploitation?
- Simplest solution: ϵ-greedy.
  - With probability $1 - \epsilon$ pick $a = \arg\max_{a'} Q(s, a')$ (i.e. greedy)
  - With probability ϵ pick any other action uniformly.
- Another idea - softmax using Q values
  - With probability $1 - \epsilon$ pick $a = \arg\max_{a'} Q(s, a')$ (i.e. greedy)
  - With probability ϵ pick any other action with probability $\propto \exp(\beta Q(s, a))$.
- Other fancier solutions exist, many leading methods use simple ϵ-greedy sampling.

Q-learning algorithm

Initialize $Q(s,a), \forall s \in \mathcal{S}, a \in \mathcal{A}(s)$, arbitrarily, and $Q(\textit{terminal-state}, \cdot) = 0$
Repeat (for each episode):
  Initialize S
  Repeat (for each step of episode):
    Choose A from S using policy derived from Q (e.g., ε-greedy)
    Take action A, observe R, S'
    $Q(S,A) \leftarrow Q(S,A) + \alpha\left[R + \gamma \max_a Q(S',a) - Q(S,A)\right]$
    $S \leftarrow S'$;
  until S is terminal

- Can prove convergence to the optimal Q^* under mild conditions.
- Update is equivalent to gradient descent on loss $||R + \gamma \max_a Q(S',a) - Q(s,a)||^2$.
- Why L_2 loss? Optimal solution is the mean which is what we are looking for!

Example:
→ https://www.datacamp.com/tutorial/introduction-q-learning-beginner-tutorial

Bootstrapping

- Another way to think about Q-learning.
- $Q(s, a)$ is the expected reward, can use Monte-Carlo estimation.
- Problem - you update only after the episode ends, can be very long (or infinite).
- Q-learning solution - take only 1 step forward and estimate the future using our Q value - bootstrapping.
  - "learn a guess from a guess"
- Q-learning is just one algorithm in a family of algorithms that use this idea.

Function approximation

- Q-learning still scales badly with large state spaces, how many states does a chess game have? Need to save the full table!
- Similar states, e.g. move all chess pieces two steps to the left, at treated as totally different.
- Solution: Instead of Q being a $S \times A$ table it is a parametrized function.
- Looking for function $\hat{Q}(s, a; \mathbf{w}) \approx Q^*(s, a)$
  - Linear functions $Q(s, a; \mathbf{w}) = \mathbf{w}^T \phi(s, a)$.
  - Neural network
- Hopefully can generalize to unseen states.
- Problem: Each change to parameters changes all states/actions - can lead to instability.
- For non-linear Q-learning can diverge.

Deep Q-learning

- We have a function approximator $Q(s, a; \theta)$, standard is neural net but doesn't have to be.
- What is the objective that we are optimizing?
- We want to minimize $\mathbb{E}_\rho[||R + \gamma \max_{a'} Q(S', a') - Q(s, a)||^2]$
  - ρ is a distribution over states, depends on θ!
- Two terms depend on Q, don't want to take gradients w.r. to $\gamma \max_a Q(S', a)$
- We want to correct our previous estimation given the new information.

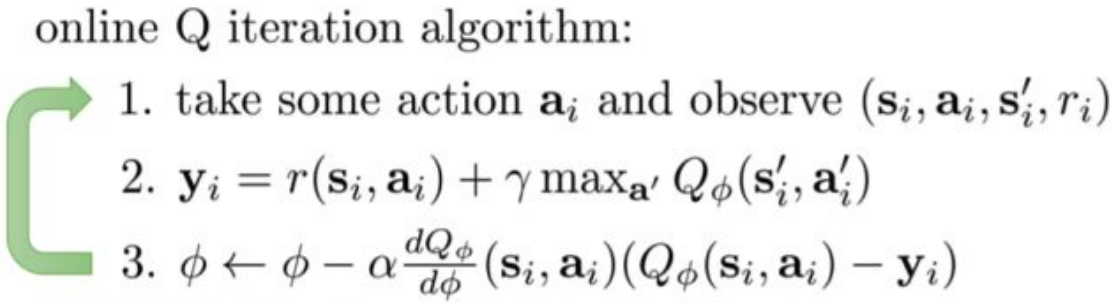


Figure: Take from:rll.berkeley.edu/deeprlcourse

- This simple approach doesn't work well as is.

Issue and solutions

- Problem: data in the minibatch is highly correlated
  - Consecutive samples are from the same episode and probably similar states.
  - Solution: Replay memory.
  - You store a large memory buffer of previous (s, a, r, s') (notice this is all you need for Q-learning) and sample from it to get diverse minibatch.
- Problem: The data distribution keeps changing
  - Since we aren't optimizing y_i its like solving a different (but related) least squares each iteration.
  - We can stabilize by fixing a target network for a few iterations

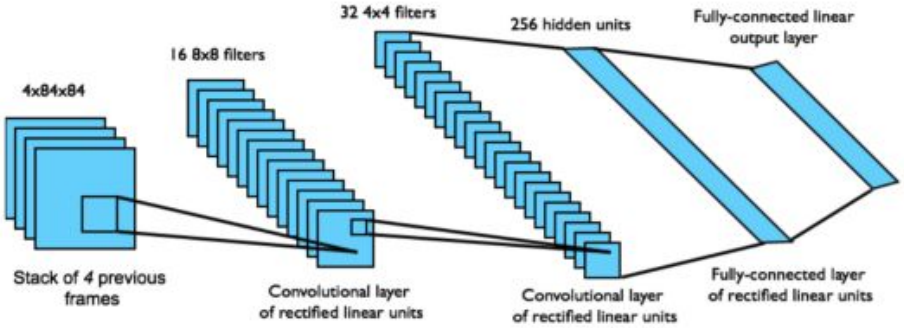


1. take some action $\mathbf{a}_i$ and observe $(\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i, r_i)$, add it to $\mathcal{B}$
2. sample mini-batch $\{\mathbf{s}_j, \mathbf{a}_j, \mathbf{s}'_j, r_j\}$ from $\mathcal{B}$ uniformly
3. compute $y_j = r_j + \gamma \max_{\mathbf{a}'_j} Q_{\phi'}(\mathbf{s}'_j, \mathbf{a}'_j)$ using *target* network $Q_{\phi'}$
4. $\phi \leftarrow \phi - \alpha \sum_j \frac{dQ_\phi}{d\phi}(\mathbf{s}_j, \mathbf{a}_j)(Q_\phi(\mathbf{s}_j, \mathbf{a}_j) - y_j)$
5. update ϕ': copy ϕ every N steps

Figure: Take from:rll.berkeley.edu/deeprlcourse

Example. DQN on Atari games

- Trained a NN from scratch on atari games



- Ablation study

	Replay Fixed-Q	Replay Q-learning	No replay Fixed-Q	No replay Q-learning
Breakout	316.81	240.73	10.16	3.17
Enduro	1006.3	831.25	141.89	29.1
River Raid	7446.62	4102.81	2867.66	1453.02
Seaquest	2894.4	822.55	1003	275.81
Space Invaders	1088.94	826.33	373.22	301.99

RL recap

- Learning from experience not from labeled examples.
- Why is RL hard?
  - Limited feedback.
  - Delayed rewards.
  - Your model effect what you see.
  - Huge state space.
- Usually solved by learning the value function or optimizing the policy (not covered)
- Model based method but less successful at the moment.
- How do you define the rewards? Can be trick.
  - Bad rewards can lead to reward hacking

Q-Learning recap

- Try to find Q that satisfies the optimal Bellman conditions
- Off-policy algorithm - Doesn't have to follow a greedy policy to evaluate it.
- Model free algorithm - Doesn't have any model for instantaneous reward or dynamics.
- Learns a separate value for each s, a pair - doesn't scale up to huge state spaces.
- Can scale using a function approximation
  - No more theoretical guarantees.
  - Can diverge.
  - Some simple tricks help a lot.

Reinforcement Learning with Q-learning and OpenAI Gym

Installing and Setting Up Gymnasium

Creating Custom Environments

Implementing Reinforcement Learning Algorithms

Visualizing Training and Results

References

1. https://www.learndatasci.com/tutorials/reinforcement-q-learning-scratch-python-openai-gym/
2. https://medium.com/swlh/introduction-to-q-learning-with-openai-gym-2d794da10f3d
3. https://github.com/srnand/Reinforcement-Learning-using-OpenAI-Gym/tree/master/Mountain_Car
4. https://gymnasium.farama.org/